Vaidehi Gohil, Anthony Yalong

Prof. Kaeli

EECE5640

March 13, 2026

**Project Proposal**

*1. Workloads*

We will evaluate two DSP workloads: Fast Fourier Transform (FFT) and FIR Filtering (Finite Impulse Response). Both are computationally intensive signal processing algorithms with well-understood parallel structure, making them well-suited for comparison across parallelization strategies.

FFT transforms a signal from the time domain to the frequency domain. The recursive divide-and-conquer structure of the Cooley-Tukey algorithm presents natural opportunities for parallelism, particularly across independent sub-transform computations.

FIR Filtering applies a linear time-invariant filter to a signal via discrete convolution. Each output sample is computed as a dot product of the filter coefficients with a window of input samples, making it embarrassingly parallel across output samples.

Studying both workloads allows us to compare the parallel scaling behavior of a divide-and-conquer algorithm (FFT) against an embarrassingly parallel one (FIR) within a coherent DSP application domain.

*2. Input Datasets*

We will use three input datasets of varying sizes to evaluate strong scaling behavior: a small 65,536-sample (2^16) synthetic audio signal, a medium 1,048,576-sample (2^20) synthetic audio signal, and a large real-world audio recording sourced from the LibriSpeech ASR corpus resampled to approximately 4,000,000 samples. The small and medium signals will be generated in-house. For FIR filtering, a low-pass filter with 1024 taps will be applied to each of the three signals, with filter coefficients generated using the Hamming window method.

*3. Platform*

We will run all experiments on the Explorer Cluster at Northeastern University using the course reservation. Jobs will be submitted via SBATCH scripts targeting Intel Cascade Lake nodes (--constraint=cascadelake), with each experiment running on a single multi-core node scaling from 1 to 32 threads or processes. Each workload will be implemented using three parallelization middlewares:

- Pthreads – manual thread management with fine-grained control
- OpenMP – directive-based shared-memory parallelism
- OpenMPI – message-passing parallelism across multiple processes on a single node

This allows us to compare not only parallel scaling behavior but also the programming effort and runtime overhead associated with each middleware.

*4. Experiments*

For each workload and middleware combination, we will measure wall-clock execution time at 1, 2, 4, 8, 16, and 32 threads or processes across all three input dataset sizes. Each configuration will be run 5 times and results will be reported as mean and standard deviation. Speedup and parallel efficiency will be computed relative to a serial baseline. This yields a total of 2 workloads x 3 middlewares x 3 input sizes x 6 thread counts = 108 experimental configurations per timing run.

*5. Performance Analysis Tools and Metrics*

Wall-clock time will be measured consistently across all three middleware implementations. Intel VTune Profiler, available as a loadable module on Explorer, will be used to collect CPU utilization, thread-level parallelism, and hotspot analysis. The primary metrics collected will be execution time, speedup S(p) = T(1) / T(p), parallel efficiency E(p) = S(p) / p, and serial fraction estimated via Amdahl's Law.

*6. Expected Results and Grading*

| Grade | Description |
|---|---|
| *A* | Both workloads implemented in all three middlewares. All three input datasets used. Timing, speedup, and efficiency reported and thoroughly analyzed for all configurations. VTune profiling included. Writeup suitable for sharing with a future employer. |
| *A-* | Both workloads implemented in all three middlewares. All three input datasets used. Results reported but analysis is less thorough. |
| *B+* | Both workloads implemented in two middlewares. Two input datasets used. Results reported and analyzed. |
| *B* | One workload implemented in two middlewares. Two input datasets used. Results reported and analyzed. |
| *C* | One workload implemented in one middleware. Results reported with minimal analysis. |
| *F* | Else. |

Extra Credit Opportunities

1. Presentation: 10-minute in-class presentation during the final two class sessions.
2. Additional profiling: Use of additional Explorer profiling tools beyond VTune (e.g., perf, TAU).
3. Weak scaling analysis: Evaluate weak scaling by increasing problem size proportionally with thread count.